API Reference

SET09102 Coursework: API Reference

API Overview

Base URL: `https://set09102-api.b-davison.workers.dev`

Authentication: JWT Bearer token (required for most endpoints)

Content Type: `application/json`

Response Format: JSON

> 💡 **Interactive Docs:** A Swagger UI with live try-it-out is available at `https://set09102-api.b-davison.workers.dev/` (the root path). No SSH tunnel or special setup required — simply point your `HttpClient` at the base URL above.

1. Authentication Endpoints

POST /auth/register

Create a new user account.

Authentication Required: No

Request Body:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@example.com",
  "password": "SecurePassword123!"
}
```

Validation Rules:

- `firstName` : Required, 1-50 characters
- `lastName` : Required, 1-50 characters
- `email` : Required, valid email format, unique
- `password` : Required, minimum 8 characters, must contain uppercase, lowercase, number, and special character

Success Response (201 Created):

```
{
  "id": 123,
  "email": "john@example.com",
  "firstName": "John",
  "lastName": "Doe",
  "createdAt": "2026-02-03T10:00:00Z"
}
```

Error Response (400 Bad Request):

```
{
  "error": "Validation failed",
  "message": "Email already exists"
}
```

▶ Click to expand: C# Implementation Example

POST /auth/token

Login and receive JWT token.

Authentication Required: No

Request Body:

```
{
  "email": "john@example.com",
  "password": "SecurePassword123!"
}
```

Success Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjMiLCJlbWFpbCI6ImpvaG5AZXhhbXBsZS5
  "expiresAt": "2026-02-10T15:30:00Z",
  "userId": 123
}
```

Error Response (401 Unauthorized):

```
{
  "error": "Authentication failed",
  "message": "Invalid email or password"
}
```

> 💡 **Implementation Note:** Store the JWT token securely (e.g., using SecureStorage in MAUI). Include it in the `Authorization` header for all authenticated requests: `Authorization: Bearer {token}`

▶ Click to expand: C# Implementation Example

GET /users/me

Retrieve current user's profile.

Authentication Required: Yes

Headers:

```
Authorization: Bearer {token}
```

Success Response (200 OK):

```
{
  "id": 123,
  "email": "john@example.com",
  "firstName": "John",
  "lastName": "Doe",
  "averageRating": 4.5,
  "itemsListed": 5,
  "rentalsCompleted": 12,
  "createdAt": "2026-01-15T10:00:00Z"
}
```

Error Response (401 Unauthorized):

```
{
  "error": "Unauthorized",
  "message": "Invalid or expired token"
}
```

GET /users/{id}/profile

View public profile of another user.

Authentication Required: No (public endpoint)

URL Parameters:

- `id` (integer): User ID

Success Response (200 OK):

```
{
  "id": 456,
  "firstName": "Sarah",
  "lastName": "Smith",
  "averageRating": 4.8,
  "itemsListed": 8,
  "rentalsCompleted": 24,
  "reviews": [
    {
      "id": 101,
      "rating": 5,
      "comment": "Great item, very helpful owner!",
      "reviewerName": "Mike Johnson",
      "createdAt": "2026-01-20T14:00:00Z"
    }
  ]
}
```

Error Response (404 Not Found):

```
{
  "error": "Not found",
  "message": "User not found"
}
```

2. Item Discovery Endpoints

GET /items

List all available items (with optional filters).

Authentication Required: No

Query Parameters:

- `category` (string, optional): Filter by category slug (e.g., "tools", "camping")
- `search` (string, optional): Search in title and description
- `page` (integer, optional): Page number (default: 1)
- `pageSize` (integer, optional): Items per page (default: 20, max: 100)

Example: `GET /items?category=tools&page=1&pageSize=20`

Success Response (200 OK):

```
{
  "items": [
    {
      "id": 789,
      "title": "Electric Drill",
      "description": "Cordless drill with battery",
      "dailyRate": 5.00,
      "categoryId": 1,
      "category": "Tools",
      "ownerId": 456,
      "ownerName": "Sarah Smith",
      "ownerRating": 4.8,
      "isAvailable": true,
      "averageRating": 4.5,
      "imageUrl": "https://example.com/images/drill.jpg",
      "createdAt": "2026-01-10T10:00:00Z"
    }
  ],
  "totalItems": 45,
  "page": 1,
  "pageSize": 20,
  "totalPages": 3
}
```

GET /items/nearby

Find items near a location using PostGIS spatial queries.

Authentication Required: No

Query Parameters:

- `lat` (float, required): Latitude (-90 to 90)
- `lon` (float, required): Longitude (-180 to 180)
- `radius` (float, optional): Search radius in kilometers (default: 5, max: 50)
- `category` (string, optional): Filter by category slug

Example: `GET /items/nearby?lat=55.9533&lon=-3.1883&radius=10&category=tools`

Success Response (200 OK):

```
{
  "items": [
    {
      "id": 789,
      "title": "Electric Drill",
      "description": "Cordless drill with battery",
      "dailyRate": 5.00,
      "categoryId": 1,
      "category": "Tools",
      "ownerId": 456,
      "ownerName": "Sarah Smith",
      "latitude": 55.9600,
      "longitude": -3.1900,
      "distance": 2.3,
      "isAvailable": true,
      "averageRating": 4.5,
      "imageUrl": "https://example.com/images/drill.jpg"
    }
  ],
  "searchLocation": {
    "latitude": 55.9533,
    "longitude": -3.1883
  },
  "radius": 10.0,
  "totalResults": 1
}
```

> 💡 **PostGIS Implementation:** This endpoint uses PostGIS `ST_DWithin` function for efficient spatial queries. The distance field is calculated in kilometers using `ST_Distance` on geography types.

Error Response (400 Bad Request):

```
{
  "error": "Validation failed",
  "message": "Invalid coordinates: latitude must be between -90 and 90"
}
```

▶ Click to expand: PostGIS Query Example

GET /items/{id}

Get detailed item information.

Authentication Required: No

URL Parameters:

- `id` (integer): Item ID

Success Response (200 OK):

```
{
  "id": 789,
  "title": "Electric Drill",
  "description": "Cordless drill with battery, includes bits and charger. Perfect for DIY projec
  "dailyRate": 5.00,
  "categoryId": 1,
  "category": "Tools",
  "ownerId": 456,
  "ownerName": "Sarah Smith",
  "ownerRating": 4.8,
  "latitude": 55.9600,
  "longitude": -3.1900,
  "isAvailable": true,
  "averageRating": 4.5,
  "totalReviews": 8,
  "imageUrl": "https://example.com/images/drill.jpg",
  "createdAt": "2026-01-10T10:00:00Z",
  "reviews": [
    {
      "id": 888,
      "reviewerId": 123,
      "reviewerName": "John Doe",
      "rating": 5,
      "comment": "Great item, owner was very helpful!",
      "createdAt": "2026-02-01T14:00:00Z"
    }
  ]
}
```

Error Response (404 Not Found):

```
{
  "error": "Not found",
  "message": "Item not found"
}
```

POST /items

Create a new item listing.

Authentication Required: Yes

Headers:

```
Authorization: Bearer {token}
Content-Type: application/json
```

Request Body:

```
{
  "title": "Camping Tent (4-person)",
  "description": "Waterproof tent, sleeps 4 comfortably. Includes carry bag and stakes.",
  "dailyRate": 15.00,
  "categoryId": 2,
  "latitude": 55.9533,
  "longitude": -3.1883
}
```

Validation Rules:

- `title` : Required, 5-100 characters
- `description` : Optional, max 1000 characters
- `dailyRate` : Required, must be > 0 and <= 1000
- `categoryId` : Required, must exist in categories table
- `latitude` : Required, -90 to 90
- `longitude` : Required, -180 to 180

Success Response (201 Created):

```
{
  "id": 999,
  "title": "Camping Tent (4-person)",
  "description": "Waterproof tent, sleeps 4 comfortably. Includes carry bag and stakes.",
  "dailyRate": 15.00,
  "categoryId": 2,
  "category": "Camping",
  "ownerId": 123,
  "ownerName": "John Doe",
  "latitude": 55.9533,
  "longitude": -3.1883,
  "isAvailable": true,
  "createdAt": "2026-02-03T10:00:00Z"
}
```

Error Response (400 Bad Request):

```
{
  "error": "Validation failed",
  "message": "Daily rate must be greater than 0"
}
```

PUT /items/{id}

Update item details (owner only).

Authentication Required: Yes (must be item owner)

Headers:

```
Authorization: Bearer {token}
Content-Type: application/json
```

URL Parameters:

- `id` (integer): Item ID

Request Body:

```
{
  "title": "Updated Title",
  "description": "Updated description",
  "dailyRate": 12.00,
  "isAvailable": true
}
```

Success Response (200 OK):

```
{
  "id": 999,
  "title": "Updated Title",
  "description": "Updated description",
  "dailyRate": 12.00,
  "isAvailable": true,
  "updatedAt": "2026-02-03T11:00:00Z"
}
```

Error Response (403 Forbidden):

```
{
  "error": "Forbidden",
  "message": "You can only update your own items"
}
```

GET /categories

List all available categories.

Authentication Required: No

Success Response (200 OK):

```
{
  "categories": [
    { "id": 1, "name": "Tools", "slug": "tools", "itemCount": 45 },
    { "id": 2, "name": "Camping", "slug": "camping", "itemCount": 28 },
    { "id": 3, "name": "Sports", "slug": "sports", "itemCount": 32 },
    { "id": 4, "name": "Electronics", "slug": "electronics", "itemCount": 19 },
    { "id": 5, "name": "Games", "slug": "games", "itemCount": 15 }
  ]
}
```

3. Rental Workflow Endpoints

POST /rentals

Initiate a rental request (Borrower side).

Authentication Required: Yes

Headers:

```
Authorization: Bearer {token}
Content-Type: application/json
```

Request Body:

```
{
  "itemId": 789,
  "startDate": "2026-02-10",
  "endDate": "2026-02-12"
}
```

Validation Rules:

- `itemId` : Required, must exist and be available
- `startDate` : Required, must be today or future date
- `endDate` : Required, must be after startDate
- Cannot rent your own item
- Item must not have conflicting approved rentals for these dates

Success Response (201 Created):

```
{
  "id": 555,
  "itemId": 789,
  "itemTitle": "Electric Drill",
  "borrowerId": 123,
  "borrowerName": "John Doe",
  "ownerId": 456,
  "ownerName": "Sarah Smith",
  "startDate": "2026-02-10",
  "endDate": "2026-02-12",
  "status": "Requested",
  "totalPrice": 10.00,
  "createdAt": "2026-02-03T12:00:00Z"
}
```

Error Response (400 Bad Request):

```
{
  "error": "Validation failed",
  "message": "Item is not available for the selected dates"
}
```

Error Response (409 Conflict):

```
{
  "error": "Conflict",
  "message": "This item already has an approved rental for these dates"
}
```

GET /rentals/incoming

List rental requests received by the current user (items they own).

Authentication Required: Yes

Headers:

```
Authorization: Bearer {token}
```

Query Parameters:

- `status` (string, optional): Filter by status ("Requested", "Approved", "Out for Rent", "Returned", "Completed")

Success Response (200 OK):

```
{
  "rentals": [
    {
      "id": 555,
      "itemId": 789,
      "itemTitle": "Electric Drill",
      "borrowerId": 123,
      "borrowerName": "John Doe",
      "borrowerRating": 4.5,
      "startDate": "2026-02-10",
      "endDate": "2026-02-12",
      "status": "Requested",
      "totalPrice": 10.00,
      "requestedAt": "2026-02-03T12:00:00Z"
    }
  ],
  "totalRentals": 1
}
```

GET /rentals/outgoing

List rental requests sent by the current user (items they want to rent).

Authentication Required: Yes

Headers:

```
Authorization: Bearer {token}
```

Query Parameters:

- `status` (string, optional): Filter by status

Success Response (200 OK):

```
{
  "rentals": [
    {
      "id": 556,
      "itemId": 790,
      "itemTitle": "Camping Tent",
      "ownerId": 456,
      "ownerName": "Sarah Smith",
      "ownerRating": 4.8,
      "startDate": "2026-02-15",
      "endDate": "2026-02-17",
      "status": "Approved",
      "totalPrice": 30.00,
      "requestedAt": "2026-02-02T10:00:00Z",
      "approvedAt": "2026-02-02T14:00:00Z"
    }
  ],
  "totalRentals": 1
}
```

GET /rentals/{id}

Get detailed rental information.

Authentication Required: Yes (must be owner or borrower)

Headers:

```
Authorization: Bearer {token}
```

URL Parameters:

- `id` (integer): Rental ID

Success Response (200 OK):

```
{
  "id": 555,
  "itemId": 789,
  "itemTitle": "Electric Drill",
  "itemDescription": "Cordless drill with battery",
  "borrowerId": 123,
  "borrowerName": "John Doe",
  "ownerId": 456,
  "ownerName": "Sarah Smith",
  "startDate": "2026-02-10",
  "endDate": "2026-02-12",
  "status": "Approved",
  "totalPrice": 10.00,
  "requestedAt": "2026-02-03T12:00:00Z",
  "approvedAt": "2026-02-03T14:00:00Z",
  "statusHistory": [
    {
      "status": "Requested",
      "timestamp": "2026-02-03T12:00:00Z"
    },
    {
      "status": "Approved",
      "timestamp": "2026-02-03T14:00:00Z"
    }
  ]
}
```

PATCH /rentals/{id}/status

Update rental status (Approve, Reject, Return, Complete).

Authentication Required: Yes (permissions depend on status transition)

Headers:

```
Authorization: Bearer {token}
Content-Type: application/json
```

URL Parameters:

- `id` (integer): Rental ID

Request Body:

```
{
  "status": "Approved"
}
```

Valid Status Transitions:

From Status	To Status	Who Can Perform	Notes
Requested	Approved	Owner only	Owner accepts the rental request
Requested	Rejected	Owner only	Owner declines the rental request
Approved	Out for Rent	System or Owner	Automatically on start date, or manually by owner
Out for Rent	Overdue	System	Automatically if not returned by end date
Out for Rent	Returned	Borrower	Borrower marks item as returned
Overdue	Returned	Borrower	Late return
Returned	Completed	Owner	Owner confirms item is in good condition

Success Response (200 OK):

```
{
  "id": 555,
  "status": "Approved",
  "updatedAt": "2026-02-04T09:00:00Z"
}
```

Error Response (400 Bad Request):

```
{
  "error": "Invalid state transition",
  "message": "Cannot transition from Approved to Rejected"
}
```

Error Response (403 Forbidden):

```
{
  "error": "Forbidden",
  "message": "Only the owner can approve rental requests"
}
```

> 💡 **State Machine Implementation:** Consider implementing the State Pattern to handle these transitions cleanly. See the coursework specification for examples.

▶ Click to expand: State Transition Validation Example

4. Trust & Feedback Endpoints

POST /reviews

Submit a rating and comment after rental completion.

Authentication Required: Yes (must be borrower, rental must be Completed)

Headers:

```
Authorization: Bearer {token}
Content-Type: application/json
```

Request Body:

```
{
  "rentalId": 555,
  "rating": 5,
  "comment": "Great item, owner was very helpful!"
}
```

Validation Rules:

- `rentalId` : Required, must exist and be Completed
- `rating` : Required, integer 1-5
- `comment` : Optional, max 500 characters
- User must be the borrower of the rental
- Cannot review the same rental twice

Success Response (201 Created):

```
{
  "id": 888,
  "rentalId": 555,
  "reviewerId": 123,
  "reviewerName": "John Doe",
  "rating": 5,
  "comment": "Great item, owner was very helpful!",
  "createdAt": "2026-02-13T14:00:00Z"
}
```

Error Response (400 Bad Request):

```
{
  "error": "Validation failed",
  "message": "Rating must be between 1 and 5"
}
```

Error Response (403 Forbidden):

```
{
  "error": "Forbidden",
  "message": "You can only review rentals you were the borrower for"
}
```

Error Response (409 Conflict):

```
{
  "error": "Conflict",
  "message": "You have already reviewed this rental"
}
```

GET /items/{id}/reviews

Fetch all reviews for a specific item.

Authentication Required: No

URL Parameters:

- `id` (integer): Item ID

Query Parameters:

- `page` (integer, optional): Page number (default: 1)
- `pageSize` (integer, optional): Reviews per page (default: 10, max: 50)

Success Response (200 OK):

```
{
  "reviews": [
    {
      "id": 888,
      "reviewerId": 123,
      "reviewerName": "John Doe",
      "rating": 5,
      "comment": "Great item, owner was very helpful!",
      "createdAt": "2026-02-13T14:00:00Z"
    },
    {
      "id": 887,
      "reviewerId": 124,
      "reviewerName": "Jane Smith",
      "rating": 4,
      "comment": "Good condition, worked well for my project.",
      "createdAt": "2026-02-10T10:00:00Z"
    }
  ],
  "averageRating": 4.7,
  "totalReviews": 8,
  "page": 1,
  "pageSize": 10,
  "totalPages": 1
}
```

GET /users/{id}/reviews

Fetch all reviews for a specific user (as owner).

Authentication Required: No

URL Parameters:

- `id` (integer): User ID

Query Parameters:

- `page` (integer, optional): Page number
- `pageSize` (integer, optional): Reviews per page

Success Response (200 OK):

```
{
  "reviews": [
    {
      "id": 888,
      "itemId": 789,
      "itemTitle": "Electric Drill",
      "reviewerId": 123,
      "reviewerName": "John Doe",
      "rating": 5,
      "comment": "Great item, owner was very helpful!",
      "createdAt": "2026-02-13T14:00:00Z"
    }
  ],
  "averageRating": 4.8,
  "totalReviews": 24,
  "page": 1,
  "pageSize": 10,
  "totalPages": 3
}
```

5. Error Handling

All API errors follow a consistent format:

Standard Error Response

```
{
  "error": "Error type",
  "message": "Human-readable error message",
  "details": {
    "field": "Additional error details (optional)"
  }
}
```

HTTP Status Codes

Status Code	Meaning	When Used
200 OK	Success	Successful GET, PUT, PATCH requests
201 Created	Resource created	Successful POST requests
400 Bad Request	Validation error	Invalid input, validation failures
401 Unauthorized	Authentication required	Missing or invalid JWT token
403 Forbidden	Permission denied	User lacks permission for action
404 Not Found	Resource not found	Item, rental, or user doesn't exist
409 Conflict	Resource conflict	Duplicate email, overlapping rentals
500 Internal Server Error	Server error	Unexpected server-side issues

Error Handling in C#

▶ Click to expand: Error Handling Example

6. API Client Implementation Guide

IApiService Interface

Create a service interface to abstract all API communication:

▶ Click to expand: IApiService Interface

ApiService Implementation

▶ Click to expand: ApiService Implementation

Dependency Injection Setup

▶ Click to expand: MauiProgram.cs Configuration

7. Testing with the API

Mock API Service for Unit Tests

▶ Click to expand: Mock API Service Example

Integration Tests with API

▶ Click to expand: Integration Test Example

8. Rate Limiting & Best Practices

> ⚠️ **Rate Limiting:**
>
> - **Maximum 100 requests per minute per user**
> - **Maximum 1000 requests per hour per user**
> - **Exceeded rate limits return** `429 Too Many Requests`

Best Practices

1. **Cache Responses:** Cache category lists and user profiles locally
2. **Pagination:** Always use pagination for list endpoints
3. **Error Handling:** Always handle network errors and API errors gracefully
4. **Token Management:** Store JWT tokens securely using SecureStorage
5. **Location Precision:** Use appropriate precision for coordinates (6 decimal places)
6. **Date Formats:** Always use ISO 8601 format (yyyy-MM-dd) for dates
7. **Base URL:** Use `https://set09102-api.b-davison.workers.dev` as the base URL in your `HttpClient`

> 💡 **Performance Tip:** Use the `nearby` endpoint instead of fetching all items and filtering client-side. The PostGIS spatial index makes this much faster.

9. Common Issues & Troubleshooting

Connection Refused / Timeout

Problem: `HttpRequestException: Connection refused` or timeout

Solution:

- Confirm you are using `https://set09102-api.b-davison.workers.dev` as the base URL (no SSH tunnel needed)
- Check your internet connection
- The API is globally available via Cloudflare Workers — no university VPN or tunnel required

401 Unauthorized

Problem: `401 Unauthorized` on authenticated endpoints

Solution:

- Check that JWT token is being sent in `Authorization` header
- Verify token hasn't expired (tokens expire after 7 days)
- Re-login to get a new token

400 Bad Request with Validation Errors

Problem: `400 Bad Request` with validation message

Solution:

- Check request body matches API specification exactly
- Verify data types (e.g., dailyRate should be decimal, not string)
- Check date formats (must be yyyy-MM-dd)
- Ensure required fields are present

PostGIS Query Returns No Results

Problem: `GET /items/nearby` returns empty list

Solution:

- Verify latitude/longitude are correct (lat: -90 to 90, lon: -180 to 180)
- Increase search radius (try 50km)
- Check that items exist in the database with valid locations

10. API Versioning

The current API version is **v1**. All endpoints are prefixed with the base URL.

Future versions will be announced via Moodle and will maintain backward compatibility for at least one semester.

> 💡 **Need Help?**
>
> - Check the main coursework specification for tutorial links
> - Post questions in the Moodle discussion forum
> - Attend office hours for API-specific issues
> - Test API endpoints with tools like Postman or curl before implementing in your app

For coursework requirements and submission guidelines, see the main coursework specification document.

Last modified: Thursday, 26 February 2026, 8:46 AM